1. **Scripting Standards:**

    a. Create always directory with name “scripts”
    b. Give a file name as a function name ex: “helloworld” and don’t give test xyz etc if you want to print Hello world!
    c. Script should end with .shell (if multiple scripts are used)

2.Linux File Permissions:

1. By default, Linux won’t apply executable permissions to file. You need to make sure to have permissions.
   Command: chmod 755 helloworld or (chmod a+x or chmod u+rwx or chmod g-x helloworld etc.)

```
[akhan@CentOS 7 Scripts]$ ls -ltr
total 8
-rw-r--r--. 1 akhan akhan 48 Jun 27 12:03 myscript.sh
-rwxr--r--. 1 akhan akhan 22 Jun 27 12:36 helloworld
[akhan@CentOS 7 Scripts]$ chmod 755 myscript.sh
[akhan@CentOS 7 Scripts]$ ls -ltr
total 8
-rwxr-xr-x. 1 akhan akhan 48 Jun 27 12:03 myscript.sh
-rwxr--r--. 1 akhan akhan 22 Jun 27 12:36 helloworld
[akhan@CentOS 7 Scripts]$ chmod g-x myscript.sh
[akhan@CentOS 7 Scripts]$ ls -ltr
total 8
-rwxr--r-x. 1 akhan akhan 48 Jun 27 12:03 myscript.sh
-rwxr--r--. 1 akhan akhan 22 Jun 27 12:36 helloworld
[akhan@CentOS 7 Scripts]$
```

2. **Script Format:**

Good Scripting format makes code readable and easy to understand.

1. **Define shebang: #!/bin/bash or #!/usr/bin/env bash (Universal for all shells)**

➔ Definition: The very first line of a Linux/Unix script that tells the kernel which interpreter to use to execute the file.
➔ Syntax: `#!` (Hash + Bang).
➔ How it works: When you execute a script, the OS reads the shebang line, loads the specified program, and passes the script to it.
➔ Example: `#!/bin/bash` tells the system to use the Bash shell to run the script.

Why use #!/usr/bin/env? – It locates the bash executable file

- Hardcoded paths fail**:** #!/bin/bash will fail if a system has Bash installed in /usr/bin/bash or /usr/local/bin/bash.
- Dynamic Search**:** env searches the user's $PATH variable and automatically finds the correct, absolute location of the interpreter on that specific machine.

2. **Comments (#)**
3. **Define Variables** (Ex name=Ameer)
4. **Commands (echo cp, mv, grep etc.)**
5. **Statements (if, for, while)**
6. **Make sure to give proper indentations for code readability**

Best Practise:

Writing a clean, well-formatted shell script saves you hours of debugging and makes your life (and your teammates' lives) much easier.
Here are the ultimate blueprint and best practices for formatting a Bash script.

1. The Standard Script Template

A professionally formatted script should generally follow this structural flow

Bash

```bash
#!/usr/bin/env bash

# ============================================================================
# Script Name: backup_data.sh
# Description: Syncs local production data to secure cloud storage.
# Author:      Your Name
# Date:        2026-06-27
# ============================================================================

# --- 1. SETTINGS & SAFETY ---
set -euo pipefail
IFS=$'\n\t'

# --- 2. CONSTANTS & ENVIRONMENT ---
readonly BACKUP_DIR="/var/log/app"
readonly DESTINATION="s3://my-bucket/backups"

# --- 3. GLOBAL VARIABLES ---
LOG_FILE=""

# --- 4. HELPER FUNCTIONS ---
log_message() {
    local message="$1"
    echo "[$(date +'%Y-%m-%d %H:%M:%S')] $message"
}

cleanup() {
    log_message "Cleaning up temporary files..."
    # Cleanup logic here
}
trap cleanup EXIT

# --- 5. MAIN LOGIC ---
main() {
    log_message "Starting backup process..."
    # Your core code goes here
}

# Invoke main function
main "$@"
```

2. Formatting Best Practices

Use the Right Shebang

Always use #!/usr/bin/env bash instead of #!/bin/bash. This ensures better portability across different Unix-like systems (like macOS and various Linux distributions), as env locates bash dynamically in the user's path.

The "Safety Header" (set -euo pipefail)

Include this right below your header. It prevents silent failures:

- set -e: Exit immediately if a command exits with a non-zero status.
- set -u: Treat unset variables as an error and exit immediately.
- set -o pipefail: Ensure that if any command in a pipeline fails, the whole pipeline returns a failure code.

Indentation and Line Length

- **Indent with Spaces:** Use **2 or 4 spaces** per indent level. Do not use hard tabs, as they render differently across different text editors and code review platforms.
- **Wrap Long Lines:** Keep lines under **80 characters** for readability. Use the backslash (\) to break long commands into multiple lines:

Variable Casing and Scoping

- **UPPERCASE** for constants and environment variables (e.g., readonly API_URL="https://...").
- **lowercase** for regular, temporary, and script-internal variables (e.g., file_count=0).
- **Always use local inside functions:** This prevents functions from accidentally overwriting global variables.

Always Quote Your Variables

To prevent word splitting and globbing issues (especially with paths that contain spaces), always wrap your variables in double quotes.

- ❌ **Bad:** rm -rf $target_dir (If $target_dir is empty or has spaces, this can be catastrophic).
- = **Good:** rm -rf "$target_dir"

Use Modern Syntax

- **Test conditions:** Use double brackets [[...]] instead of old single brackets [...]. Double brackets are safer, don't require quoting variables inside them, and support advanced regex matching.

- **Command Substitution:** Use $(command) instead of backticks `command`. It supports nesting and is much easier to read.

3. Recommended Linting Tool

Before you run any script, run it through **ShellCheck**. It is a static analysis tool that will instantly point out formatting issues, syntax errors, and security vulnerabilities in your script. Most modern IDEs (like VS Code) have a ShellCheck extension that lints your code in real time.

3. **Script Sequence:**

Script always executes command in a sequential order like command 1, command2 etc.

Example: ls, pwd, etc.